

SOFTAIMS · TOOLING ANALYSIS

Where JMeter hurts. Where k6 helps.

A direct comparison of documented weaknesses and measured strengths — with a source or a measurement behind every single claim.

k6 v2.2.0 · JMeter 5.6.3 September 2026

Rules this deck holds itself to

A one-sided argument only works if every part of it survives being checked. So:

What you will see

- **Measured** — numbers we produced on our own hardware, reproducible from our repository
- **Documented** — limitations Apache states in JMeter's own manual
- **Registered** — security issues with published CVE identifiers and severity scores
- Every slide carries its source

What you will not see

- No claim that JMeter is broken — it is not; it is mature and capable
- No security issue presented as current when it has been patched
- No invented benchmark figures
- Slide 18 lists where JMeter genuinely beats k6

WHY THE COUNTER-ARGUMENT IS INCLUDED

A deck with no weaknesses on its own side gets dismissed as marketing. The fastest way to lose this argument is to have somebody in the room name a JMeter advantage we failed to mention.

Three critical remote-code-execution flaws, all in the same feature

CVE	SCORE	PUBLISHED	WHAT IT ALLOWED
CVE-2019-0187	9.8	Mar 2019	Unauthenticated remote code execution via RMI connection and untrusted deserialisation
CVE-2018-1297	9.8	Feb 2018	Unsecured RMI connection in distributed mode
CVE-2018-1287	9.8	Feb 2018	RMI registry bound to wildcard host — unauthorised code execution

The common thread

All three are in **distributed mode** — the feature you must use once one machine is not enough. All three score **9.8 out of 10**, the highest practical severity.

The cause is **Java RMI** — *an old mechanism for one Java program to run code inside another over the network*, which JMeter uses to link controller and workers.

STATED HONESTLY

All three are patched. JMeter 5.1 and later add encryption and node authentication, and 5.6.3 is not vulnerable. Anyone presenting these as live risks is misleading you.

THE POINT THAT SURVIVES THE PATCH

Scaling JMeter means exposing **RMI ports 1099 and 50000** between machines — an architecture that produced three critical RCEs. k6 has **no CVEs** in the national vulnerability database, and scales through Kubernetes rather than RMI.

Its most-loved feature cannot be used for its main job

APACHE JMETER'S OWN USER MANUAL

“JMeter is **not** designed to produce high loads in GUI mode.” Running load tests from the interface “will not only freeze but also consume loads of resources and produce **unreliable load test results.**”

The contradiction

JMeter's headline advantage over k6 is the visual builder — the reason a non-coder can start quickly.

But that interface may only be used for **authoring and debugging**. Every real measurement must run headless:

```
jmeter -n -t test.jmx -l results.jtl
```

So the beginner who chose JMeter *for the GUI* still has to learn the command line.

Why the GUI distorts results

- Rendering graphs and tables consumes the same CPU that should be generating load
- Listeners hold every sample in memory
- The interface freezes at high thread counts
- Inflated response times look like an application problem

Listener — a JMeter component that collects and displays results — cheap in a small test, ruinous in a large one

Source: jmeter.apache.org/usermanual/best-practices.html

It ships configured to run out of memory

The shipped default

```
HEAP="-Xms1g -Xmx1g  
-XX:MaxMetaspaceSize=256m"
```

Verified in `jmeter.bat` on our own installation of 5.6.3. The memory ceiling is fixed at **1 GB** regardless of the machine's actual capacity.

WHAT HAPPENS NEXT

Push past a few hundred threads and the test dies with **OutOfMemoryError** — on a machine with 16 or 32 GB free. The fix requires knowing to edit a startup script that nothing tells you about.

Measured on our hardware, 1,000 users

JMeter		841 MB
k6		320 MB

Per virtual user: 0.76 MB against 0.17 MB — a 4.4× difference in marginal cost.

THE WIDER PATTERN

Memory exhaustion is common enough in JMeter practice that vendors publish dedicated troubleshooting guides for it. It is not an exotic failure — it is the expected outcome of the default configuration meeting a realistic test.

Source: `measured` — `benchmarks/` • heap default verified in `jmeter.bat` 5.6.3

One operating-system worker for every simulated person

JMETER WORKERS

1,037

for 1,000 simulated users

K6 WORKERS

15

for the same 1,000 users

DIFFERENCE

69×

measured, not estimated

Why this is structural, not a setting

A simulated user spends about **98% of its life waiting** — for a response, or for think time to pass. JMeter reserves a full operating-system worker for that waiting anyway.

Beyond a few hundred workers, the machine spends real processor time simply *switching between them* rather than generating load.

Industry guidance is **200–300 threads per JMeter engine**. Past that you need more machines — which returns you to the RMI distributed mode from slide 3.

How k6 avoids it

Because users are mostly idle, a small pool of workers can serve many of them by taking turns. Grafana's own documentation puts a single k6 process at **30,000–40,000 virtual users**.

The consequence for Softaims is direct: **the load a whole JMeter cluster needs, k6 often runs on one build server**.

Source: measured — benchmarks/ · BlazeMeter thread guidance · grafana.com/docs/k6 large tests

Tests nobody can actually review

What a JMeter test looks like on disk

```
<stringProp name="ThreadGroup
  .num_threads">1000</stringProp>
<elementProp name="HTTPSampler
  .Arguments" elementType="Argum...
<boolProp name="LoopController
  .continue_forever">false</boolProp>
```

Around **800 lines** of machine-generated **XML** — *a text format written by the tool for the tool* — technically readable, practically not for a test a person would describe in three sentences.

WHAT THIS COSTS DAY TO DAY

- **Reviews become rubber stamps.** A colleague approves because it ran, not because they read it
- **Merge conflicts are brutal.** Two testers editing one plan produces XML nobody can reconcile by hand
- **Change history is unreadable.** A diff shows moved XML nodes, not intent
- **Copy-paste replaces reuse.** There is no clean way to share logic between plans

THE SAME TEST IN K6

About **50 lines of JavaScript** — reviewed in a pull request like any other change, by people who already read that language every day.

Source: generated test plans from our own benchmark — benchmarks/test.jmeter.jmx

Two ways it misreports what it measures

A · Silent result corruption

Leave a listener enabled during a load run and JMeter's own overhead inflates the numbers it is recording.

The symptom looks exactly like a slow application. Teams have chased phantom performance bugs that lived entirely in the test configuration.

Nothing warns you. The test simply reports worse numbers.

B · It under-reports the worst moments

Apache's own manual warns that incorrect thread sizing produces **coordinated omission** — *the test stops sampling during exactly the periods the system is slowest, so the report flatters it.*

The result is a report that looks survivable for a system that would collapse in production.

Measured against a known truth

We built a target that always takes exactly **50 ms**, then asked each tool to measure it:

JMeter		61.7 ms
k6		56.8 ms
truth		50.0 ms

FAIR READING OF THAT CHART

Neither tool lied — the excess is real time spent inside each one's HTTP client. But JMeter's error was **11.7 ms against k6's 6.8 ms**, and its worst single reading was **206 ms** against k6's **108 ms**.

On a fast API that gap decides whether you meet your target. On a 300 ms API it is noise — we are not claiming otherwise.

No shipped release in two and a half years

JMETER LATEST RELEASE

5.6.3

9 January 2024

K6 LATEST RELEASE

v2.2.0

10 August 2026

COMMUNITY SIZE

3.3×

31,368 stars against 9,523

What stalled delivery costs

- New protocols and platform features arrive slowly or not at all
- Fixes reach you only via third-party plugins with their own release cycles
- Fewer new practitioners means a smaller hiring pool over time
- Modern workflows — containers, cloud runners, AI tooling — are not being built into it

STATED FAIRLY

The project is not abandoned. The repository had a commit in the last fortnight, and Apache governs it. The gap is between *activity* and *delivery* — commits happen, releases do not.

THE CONTRAST

In the same period k6 shipped two major versions and now maintains **two release lines in parallel** — v1.8.x for stability and v2.x for new capability.

Source: GitHub API, retrieved 2 September 2026

The weaknesses, and what each one costs us

#	WEAKNESS	COST TO SOFTAIMS	EVIDENCE
1	Three 9.8-severity RCE flaws in distributed mode	RMI ports exposed between machines when scaling	NVD
2	GUI unusable for real load tests	Its main selling point does not apply to the main job	Apache docs
3	Fixed 1 GB heap by default	Out-of-memory crash at a few hundred users	Verified
4	One OS worker per virtual user	1,037 workers vs 15; needs a cluster far sooner	Measured
5	Tests stored as 800 lines of XML	Reviews become rubber stamps; merges are painful	Our plans
6	Listeners corrupt results; omission under-reports the tail	Phantom bugs, and false confidence	Apache docs
7	No release since January 2024	Modern workflows are not arriving	GitHub API

READ THIS CORRECTLY

None of these makes JMeter unusable — teams run it successfully every day, and most items have a workaround a skilled engineer already knows. The argument is about **where the friction sits**: with JMeter you spend effort managing the tool, and that effort scales with the size of your test.

PART TWO

What k6 does genuinely well

Six strengths, each one either measured on our hardware or demonstrated on the k6 already installed on this machine.

It leaves the machine free to do its actual job

MEMORY

2.6×

less — 320 MB against 841 MB

WORKERS

69×

fewer — 15 against 1,037

PER VIRTUAL USER

4.4×

cheaper — 0.17 MB against 0.76 MB

What it means in machines

To simulate 10,000 users on a
4 GB build server:

```
k6      → 1 machine
JMeter  → 3 machines
          + splitting the test
          + aligning their clocks
          + merging the results
```

WHY EFFICIENCY IS REALLY ABOUT TRUST

A load generator that runs out of resources **cannot send the traffic it promised**. The server never receives the intended load, and every number reported is wrong — flatteringly so.

Headroom on the generator is not a cost saving. It is the precondition for the result meaning anything at all.

Source: measured — benchmarks/ • machine count is a memory-only calculation

The performance requirement lives inside the test

Written once, checked forever

```
thresholds: {  
  http_req_duration: ['p(95)<500'],  
  http_req_failed:   ['rate<0.01']  
}
```

In plain words: *95% of requests must finish under half a second, and fewer than 1% may fail.*

If the rule breaks, k6 exits with a failure code and the pipeline stops. No plugin, no glue, no separate parsing step.

The loop this creates

```
developer pushes code  
↓  
test runs  
↓  
threshold broken  
↓  
build blocked
```

WHY THIS IS THE STRONGEST SINGLE ARGUMENT

A slowdown caught here costs minutes. The same slowdown found by a customer costs an incident. This turns performance from an *event* somebody schedules into a **property the pipeline enforces**.

Source: grafana.com/docs/k6/latest/using-k6/thresholds/

It tells you when the test itself failed

Arrival-rate scenarios

```
constant-arrival-rate  
ramping-arrival-rate
```

Requests start on a clock whether or not earlier ones have returned — which is how real customers behave. This is what avoids the under-reporting on slide 8.

dropped_iterations

```
dropped_iterations....: 1,204
```

A metric that says: “**I could not start this much of the work you asked for.**”

The generator confessing its own shortfall — the single most valuable signal for deciding whether to trust a run.

WHY THIS MATTERS MORE THAN ANY SPEED COMPARISON

Most bad load-testing decisions come from a test that *quietly* failed to apply the load it claimed. Both tools can produce that failure. **k6 makes it visible by default** — and a visible failure is one you can act on.

Source: grafana.com/docs/k6/ — scenarios and arrival-rate VU allocation

Modern tooling that is already on this machine

Verified by running it here

```
$ k6 x
```

agent	Bootstrap an AI-assisted k6 workflow in any editor
mcp	An MCP server for k6 for AI agents
docs	CLI k6 docs for agents and users
explore	Explore k6 extensions

Not a roadmap item — these ran on our installed k6 v2.1.0.

Automatic extension resolution

```
Automatic extension resolution  
is enabled...  
Downloading custom k6 binary...  
Custom k6 binary ready
```

Import something k6 does not yet have, and it **builds itself a binary that does**. We watched this happen while writing this deck.

JMeter's equivalent is downloading a plugin JAR and managing its dependencies by hand.

ALSO BUILT IN

Live dashboard

A real-time chart view and an exportable HTML report, with no plugin

ALSO BUILT IN

Grafana output

Metrics stream to Prometheus and Grafana for correlation with server telemetry

ALSO BUILT IN

Browser mode

Drives real Chromium for front-end timing JMeter cannot measure at all

Source: executed on our own k6 v2.1.0 installation · grafana.com/docs/k6

Tests that behave like the rest of our code

A complete k6 test

```
export default function () {
  const res = http.get(url);
  check(res, {
    'status is 200': r => r.status === 200
  });
  sleep(1);
}
```

Everything that follows from it

- Reviewed in a **pull request**, by people who read JavaScript daily
- Diffs show **intent**, not moved XML nodes
- Shared logic lives in imported modules, not copy-paste
- Merge conflicts resolve like any other code conflict
- The test lives beside the service it tests

THE THREE-YEAR QUESTION

For one engineer writing one test this week, JMeter's builder is quicker. For **a team maintaining a suite for three years**, the file has to be readable by whoever inherits it. That is the horizon Softaims is buying for.

Source: our own generated test plans — benchmarks/test.k6.js and test.jmeter.jmx

SIDE BY SIDE – SHADED ROWS ARE JMETER WINS, COVERED NEXT

The comparison in one table

DIMENSION	JMETER 5.6.3	K6 V2.2.0	EVIDENCE
Worker per virtual user	1 OS thread each — 1,037	shared pool — 15	measured
Memory, 1,000 users	841 MB	320 MB	measured
Memory per user	0.76 MB	0.17 MB	measured
Reported latency vs 50 ms truth	61.7 ms	56.8 ms	measured
Worst single reading	206 ms	108 ms	measured
Default memory ceiling	fixed 1 GB — OOM risk	grows with the machine	verified
Test stored as	~800 lines XML	~50 lines JavaScript	our plans
Pass/fail gates for CI	plugins and glue	built-in thresholds	docs
Warns when it under-delivers load	no equivalent metric	dropped_iterations	docs
GUI usable for load runs	no — Apache advises against	n/a — code only	Apache docs
Registered CVEs	3 rated 9.8 (all patched)	none found	NVD
Latest release	January 2024	August 2026	GitHub
Community size	9,523 stars	31,368 stars	GitHub
Protocol breadth	JDBC, JMS, LDAP, FTP, SMTP	HTTP, gRPC, WS, browser	docs
Visual test builder	yes	no	—
Free distributed testing	built in	needs Kubernetes or cloud	docs

Where JMeter is genuinely the better tool

If we present only the criticism, the first person to name one of these wins the argument. So here they are, unprompted.

DECISIVE

Protocol breadth

JDBC, JMS, LDAP, FTP, SMTP, POP3, IMAP, raw TCP. k6 cannot speak most of these at any level of effort.

If we need to load-test a database or a message queue directly, JMeter is the answer — not a compromise.

REAL

Free distributed testing

Controller and worker mode is built in and runs on ordinary virtual machines.

k6's equivalent needs a **Kubernetes cluster** or the paid cloud service — a genuine gap for anyone without either.

REAL

Non-coders and stability

The visual builder and proxy recorder let a QA colleague produce a working test without writing code.

And JMeter **does not break its own API** — k6 v2 removed features that existing scripts relied on.

THE HONEST SUMMARY

JMeter is **mature, capable and free**, with two decades of plugins and expertise behind it. Every criticism in this deck is about fit for *our* workload — not about quality. A company testing mostly databases and queues should choose JMeter, and would be right to.

What follows from all of it

DEFAULT FOR NEW WORK

k6

- Uses **2.6× less memory** and **69× fewer workers**
- Reports latency **closer to the truth**, and tells you when it under-delivers load
- Pass/fail gates built in — performance becomes a pipeline property
- Tests a colleague can actually review
- Actively released; three times the community

KEPT FOR WHAT ONLY IT DOES

JMeter

- Databases, message queues, directory and mail servers
- Existing suites that already work — **no migration project**
- Colleagues who need a visual builder
- Distributed load without a Kubernetes cluster

k6 default JMeter specialist

Seven documented weaknesses, six measured strengths, and three honest concessions. The conclusion is not that JMeter is bad — it is that **the friction sits in different places**, and for the API and cloud work Softaims actually does, k6's friction is smaller.

Every claim, and where it came from

Measured by us

- 1,000 virtual users against a target of known, fixed 50 ms latency. Memory, workers and CPU sampled every 500 ms. Both runs 0% errors.
`benchmarks/` — `harness`, `raw results`, `README`
- Windows 11 · 8 logical cores · 15.6 GB RAM · k6 v2.1.0 · JMeter 5.6.3 · OpenJDK 17
- JMeter default heap confirmed in `jmeter.bat` of our own 5.6.3 install
- k6 `x` subcommands and automatic extension resolution executed on this machine

Security

- CVE-2019-0187 — 9.8 critical, unauthenticated RCE via RMI, March 2019
- CVE-2018-1297 — 9.8 critical, unsecured RMI connection
- CVE-2018-1287 — 9.8 critical, RMI registry bound to wildcard host
- **All patched;** upgrade to 5.1 or later resolves them
`services.nvd.nist.gov` — National Vulnerability Database

Apache JMeter documentation

- Best practices — non-GUI mode, listener overhead, thread sizing, coordinated omission
`jmeter.apache.org/usermanual/best-practices.html`
- Elements of a test plan — protocol coverage
`jmeter.apache.org/usermanual/test_plan.html`

Grafana k6 documentation

- Thresholds as pass/fail criteria
`grafana.com/docs/k6/latest/using-k6/thresholds/`
- Open and closed models; arrival-rate VU allocation
`grafana.com/docs/k6/latest/using-k6/scenarios/concepts/`
- Running large tests — 30,000–40,000 VUs per instance
`grafana.com/docs/k6/latest/testing-guides/running-large-tests/`

Repository statistics

- GitHub API, 2 September 2026 — k6 31,368 stars, JMeter 9,523; releases August 2026 and January 2024
`api.github.com/repos/grafana/k6` · `/apache/jmeter`
- BlazeMeter — 200–300 threads per JMeter engine guidance